Test report 6 – Projects, parts, include/use

Language, engine, CLI and browser UI · 2026-10-23 · Commit e4ee224

Phase D (Playwright)	Phase A–C (regression)	Vitest	cargo test --workspace	include/use against OpenSCAD
9 / 9 passed	**56 / 56**	**167 / 167**	**274 / 274**	**4 / 4**

1 Scope

- **`include <file>` / `use <file>`** with OpenSCAD semantics: include inserts the statements of the file at its position (definitions, variables, geometry; the last assignment wins, with a warning across the file boundary); use only makes modules and functions visible, which run in their own file (they see that file's variables and its own use libraries, not transitively); own definitions take precedence over used ones, among several use the last one wins. Paths relative to the including file (or absolute). Cycles and missing files are errors at the include/use line.
- **File IDs in source positions:** each loaded file occupies its own range of positions (file f from $f{\cdot}2^{24}$, statement IDs from $f{\cdot}2^{22}$); diagnostics, hints, handles, spans and text edits thus know their file. The protocol outputs positions local to the file (`SrcRange.file`, `Diagnostic.file`, `Edit.file`, `Scene.files`); writing back hits the correct file.
- **File resolver:** `cad-eval` reads project files via the trait `FileResolver` (file system for the CLI, map for session/Wasm). The session receives the files with `SetFiles`/`SetFile`. The engine cache is unchanged (keys from the content).
- **Projects in the UI:** files as tabs above the editor (separate undo history per file), selectable main file (★), new files, removal; open a folder (File System Access API, saving writes back), otherwise upload and download a ZIP; autosave of the whole project to IndexedDB; example projects in the examples menu. Ctrl+click on the path in include/use opens the file; a double-click on an instance of a module from another file opens that file at the definition; handles write into the file of the literal (also files that are not displayed, one undo step there); face from another file: the status bar names file and line, a click opens it.
- **Multiple parts** (open item 2, plan 12.9): named instances at the top level of the main file are parts. Scene with bounding box and volume per part; parts bar with visibility and export STL/3MF/STEP per part, all as ZIP (STL or STEP) or one 3MF with one object per part. STEP (exact B-rep, AP242/AP214) goes through the same writer as the whole model: `Engine::export_step` is `Engine::export_step_nodes` of the root, parts are passed with their placement (`display_world`) via `Kernel::transform` and `Kernel::brep` to `cad_step::write`. CLI: `cad export --part name` (also `.step`, `--schema`), `--all-parts -o ordner/ [--format step]`.
- **Named instances in loops** (plan 12 item 2): `for (i = [0:3]) k[i]: halter();` → `k[2].anker`; integer index, visibility as for named instances, snapping writes `k[2].ecke`. `k: …` in a loop remains an error.
- **Example project** `examples/projekt-gehaeuse/`: gehaeuse.scad (main file), konstanten.scad (include), lib/teile.scad (use, with screw bosses `dom[i]`), parts gehaeuse and deckel.

2 Test environment

Operating system	Linux 6.18.44-fc-v153 (x64)
CPU	Intel(R) Xeon(R) Processor @ 2.10GHz · 4 cores
Node.js	v22.22.0

Browser	Chromium 141.0.7390.37 (headless, Playwright 1.56.1), WebGL via SwiftShader
Window	1440 × 900 px, device pixel ratio 1
Build	Vite 8.3.5 production build; React 19.3.1, three.js 0.186.1, CodeMirror 6.43.15
OpenSCAD	OpenSCAD version 2021.01
Source	Branch worktree-agent-a7438d365a2b87939, commit e4ee224

3 include/use against OpenSCAD 2021.01

For each project in `tests/openscad-compat/projects/`, the ECHO lines of `cad eval --json main.scad` are compared against `expected.echo` (generated with `openscad -o x.echo main.scad`) and against a fresh OpenSCAD run. The same comparison runs as the Rust test `cad-eval/tests/project_openscad.rs`.

Project	Files	Echo lines	expected.echo	OpenSCAD now
include_order	`a.scad` `b.scad` `c.scad` `lib/inner.scad` `main.scad`	4	equal	equal
lib_children	`main.scad` `werkzeug.scad`	5	equal	equal
nested_paths	`lib/konst.scad` `lib/teile.scad` `main.scad` `shared/basis.scad`	6	equal	equal
use_basics	`a.scad` `b.scad` `lib/inner.scad` `main.scad`	6	equal	equal

include_order – main.scad and echo (identical for OpenSCAD and us)

```
use <a.scad>
module m() echo("own");
m();
x = 5;
include <c.scad>
echo(x=x, y=y);
cm();
echo(f());
```

```
ECHO: "own"
ECHO: x = 1, y = 10
ECHO: "cm", 1
ECHO: "b"
```

lib_children – main.scad and echo (identical for OpenSCAD and us)

```
// Library modules with children, $-variables from
the caller, functions calling each other
// and recursion inside the library.
use <werkzeug.scad>
reihe(3, 5) sphere(1);
echo(summe([1, 2, 3, 4]));
echo(abstand(), $fn = 7);
fn_test($fn = 12);
echo(versatz = versatz);
```

```
ECHO: n = 3, kinder = 1
ECHO: 10
ECHO: 6, $fn = 7
ECHO: fn = 12, v = 3
ECHO: versatz = undef
```

nested_paths – main.scad and echo (identical for OpenSCAD and us)

```
// Relative paths: include in a subfolder, which
includes from a sibling folder; a used
// library that includes constants relative to
itself.
include <lib/konst.scad>
use <lib/teile.scad>
echo(wand = wand, rand = rand);
echo(teil_breite(2));
teil();
echo(innen = innen);
for (i = [0:1]) zeige(i);
```

```
ECHO: wand = 3, rand = 0.25
ECHO: 106
ECHO: teil = 3, innen = 100
ECHO: innen = 4
ECHO: i = 0, basis = 1.5
ECHO: i = 1, basis = 2.5
```

use_basics – main.scad and echo (identical for OpenSCAD and us)

```
use <a.scad>
use <b.scad>
m();
echo(f());
echo(getla());
echo(la=la);
showla($fn=5);
callinner();
inner();
```

```
ECHO: "b"
ECHO: "b"
ECHO: 1
ECHO: la = undef
ECHO: la = 1, fn = 5
ECHO: "inner", 7
```

4 Scenarios (Playwright, web/e2e/projekte.spec.ts)

ID	Scenario	Result	Duration	Checked
P01	Open example project: files as tabs, main file, parts	passed	2.3 s	• tabs gehaeuse.scad (active, ★), konstanten.scad, lib/teile.scad • Scene.files: main, include, use; no errors or warnings • parts gehaeuse and deckel; bounding box of the lid from y = 50
P02	Open folder (File System Access API), upload and download ZIP	passed	3.1 s	• 3 files, main file gehaeuse.scad, notizen.txt ignored • Ctrl+S writes 3 files (konstanten.scad with change) • ZIP with 3 entries; after upload the same files and parts
P03	Switch files: tabs, context from the library, undo per file	passed	3.7 s	• context gehaeuse from lib/teile.scad • change is kept when switching • undo per file
P04	Ctrl+click on an include/use path opens the file	passed	3.1 s	• Ctrl+click on <konstanten.scad> → konstanten.scad active • Ctrl+click on <lib/teile.scad> → lib/teile.scad active • a plain click does not switch
P05	Handle changes the included file (variable from include, literal in the library)	passed	10.2 s	• edit with file = mass.scad, main file unchanged • bounding box = new value • Ctrl+Z in mass.scad → breite = 40 • radius handle in lib/teile.scad writes to lib/teile.scad

P06	Hover and click across file boundaries, double-click opens the library module	passed	3.6 s	• hover → status-hover-file with file and line • click → gehaeuse.scad at the line of the face • double-click → lib/teile.scad line 15, context gehaeuse
P07	Parts list: visibility and export per part (STL, 3MF, STEP, ZIP)	passed	3.4 s	• checkbox off → plan without base, only gehaeuse • deckel.stl valid, min y = 50 • ZIP: gehaeuse.stl, deckel.stl • 3MF: 2 objects with names • deckel.step: one body, product deckel • ZIP (STEP): gehaeuse.step, deckel.step • STEP export of the whole model
P08	k[i]: anchors of the instances in the loop visible, snapping writes k[2].oben	passed	9.7 s	• parts k[0] … k[3] • 4 anchors with text k[i].oben • snap text k[2].oben while dragging • text translate(k[2].oben) sphere(2);
P09	Errors in included files: missing file at the line, syntax error in the library	passed	2.2 s	• „Datei fehlt.scad nicht gefunden“ (file fehlt.scad not found) in main.scad line 2 • syntax error with file = lib.scad • tabs with an error dot

Regression in the same run (phase A–C: ui.spec.ts, ui-b.spec.ts, ui-c.spec.ts): 56 / 56 passed.

5 Unit, Rust and CLI tests

Vitest file	passed	Failed
web/src/editor/decorations.test.ts	7 / 7	–
web/src/editor/editGroup.test.ts	7 / 7	–
web/src/editor/lang.test.ts	47 / 47	–
web/src/engine/transport.test.ts	2 / 2	–
web/src/logic/context.test.ts	5 / 5	–
web/src/logic/curves.test.ts	5 / 5	–
web/src/logic/display.test.ts	23 / 23	–
web/src/logic/drag.test.ts	12 / 12	–
web/src/logic/files.test.ts new	8 / 8	–
web/src/logic/format.test.ts	5 / 5	–
web/src/logic/grids.test.ts	5 / 5	–
web/src/logic/handles.test.ts	12 / 12	–
web/src/logic/params.test.ts	7 / 7	–
web/src/logic/positions.test.ts	5 / 5	–
web/src/logic/sceneIndex.test.ts	9 / 9	–
web/src/logic/zones.test.ts	4 / 4	–
web/src/files/zip.test.ts new	4 / 4	–

Rust test program (cargo test --workspace)	passed	new tests
tests/cli.rs	9 / 9	–
tests/projects.rs new	3 / 3	`diagnostics_name_their_file_and_line`, `check_and_eval_read_included_and_used_files`, `export_single_parts_and_all_parts`
src/lib.rs	7 / 7	–
tests/draft.rs	3 / 3	–
tests/engine.rs	25 / 25	–
tests/spec_examples.rs	4 / 4	–
tests/expr.rs	25 / 25	–
tests/grids.rs	14 / 14	–
tests/handles.rs	26 / 26	–
tests/limits.rs	8 / 8	–
tests/openscad_compat.rs	1 / 1	–
tests/openscad_deviations.rs	10 / 10	–
tests/project.rs new	16 / 16	`include_assignments_follow_textual_order`, `drag_writes_into_the_included_file`, `diamond_include_loads_the_file_twice`, `diagnostics_of_library_code_know_their_file`, `include_cycle_is_an_error`, `missing_file_is_an_error_at_the_line`, `include_inserts_definitions_variables_and_geometry`, `module_of_a_used_library_as_context`, `indexed_named_instance_errors`, `own_definitions_win_and_includes_bring_their_uses`, `paths_are_relative_to_the_including_file`, `indexed_named_instances_in_loops`, `single_file_eval_reports_includes_as_missing`, `parts_are_top_level_named_instances`, `use_imports_modules_and_functions_only`, `use_without_geometry_and_used_once`
tests/project_openscad.rs new	2 / 2	`projects_match_openscad_reference`, `projects_match_openscad_live`
tests/tree.rs	24 / 24	–
src/lib.rs	5 / 5	–
tests/render.rs	4 / 4	–
tests/project.rs new	5 / 5	`missing_files_and_syntax_errors_of_other_files`, `files_ranges_and_parts`, `set_default_writes_into_the_library`, `drag_edit_targets_the_included_file`, `export_parts`
tests/protocol_ts.rs	1 / 1	–
tests/session.rs	8 / 8	–
tests/watchdog.rs	2 / 2	–
src/lib.rs	3 / 3	–
tests/writer.rs	6 / 6	–

src/lib.rs	19 / 19	`tests::file_regions`, `tests::include_path_span`, `tests::indexed_named_instance`
tests/openscad_compat.rs	1 / 1	–
tests/spec_models.rs	1 / 1	–
tests/tree_bounds.rs	1 / 1	–
tests/brep.rs	5 / 5	–
tests/determinism.rs	1 / 1	–
tests/handles.rs	1 / 1	–
tests/kernel.rs	33 / 33	–
tests/coverage.rs	1 / 1	–

New in Rust: projects (resolver, include/use, cycle, missing file, use without geometry, writing back into an included file, context module from a library, parts, `k[i]` with error cases), OpenSCAD comparison, session (SetFiles/SetFile, file in ranges and edits, part export), CLI (`check`/`eval` with file system, diagnostics with path, `export --part`, `--all-parts`). OpenSCAD comparison of the single files (compare.py): `33/33 files match`. STEP check with OpenCASCADE over all examples (`tests/step/verify.py`, venv-step): `46/46 passed`. STEP per part, as ZIP and with `--all-parts --format step`: Rust tests `cad-session/tests/project.rs` (`export_parts`) and `cad-cli/tests/projects.rs`; in the browser P07.

6 Deviations and limitations

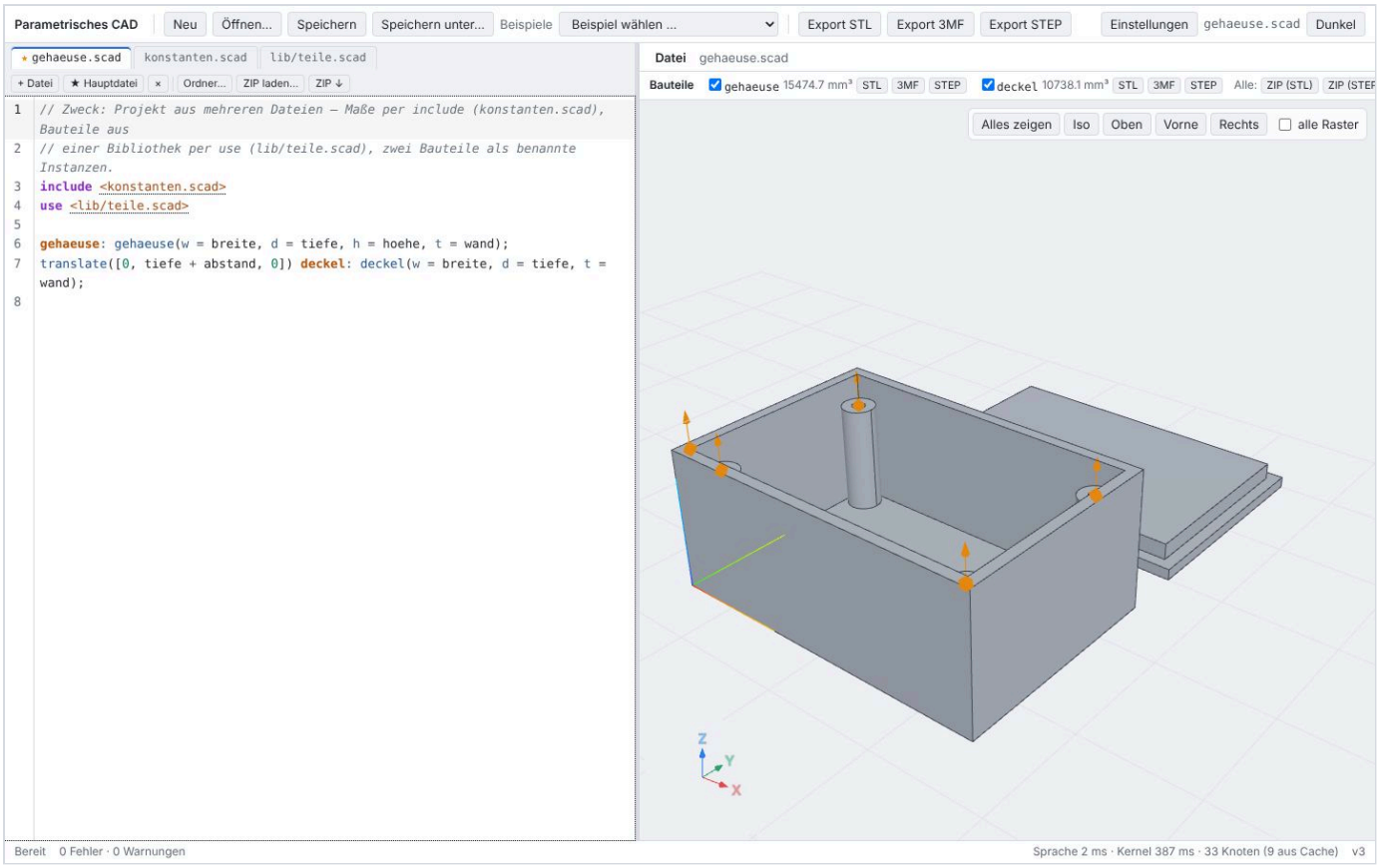
- **Search path:** only relative to the including file or absolute; no `OPENSCADPATH`/library folder. Missing files and include cycles are errors (OpenSCAD: warning, the file is skipped).
- **use:** variables of a library are evaluated once per evaluation (lazily), not anew on every call as in OpenSCAD; `$` variables at the top level of a library apply in its modules. The geometry of used files is not generated.
- **A file included twice** (via two include paths) is loaded twice (two file IDs, same path); at most 255 files, at most 16 MiB per file.
- **Snap visibility** of a grid/instance named in an included file extends to the end of that file (not into the including file); evaluating `k.anker` is not affected by this.
- **Parts** only in file context; a hidden part inside a boolean operation cannot be hidden separately (the subtree stays whole). Assemblies are open.
- **STEP of several parts in one file:** one product (name: part names joined with " + ") with all bodies, no assembly structure per part; a part whose module is an implicit union consists of several bodies (e.g. an enclosure with screw bosses).
- **solidcore** is now opt-in (`--features solidcore`); `cad diff-kernels` without the feature reports "build with `--features solidcore`" (exit 2).
- **Module names** that are the same in the main file and a library: the context (parameter bar) takes the module of the main file.

7 Screenshots and code excerpts

Largest image file: 155 KiB (JPEG, quality 80).

P01 Open example project

PROJEKT-GEHAEUSE/: THREE FILES AS TABS, MAIN FILE ★, PARTS BAR



CODE

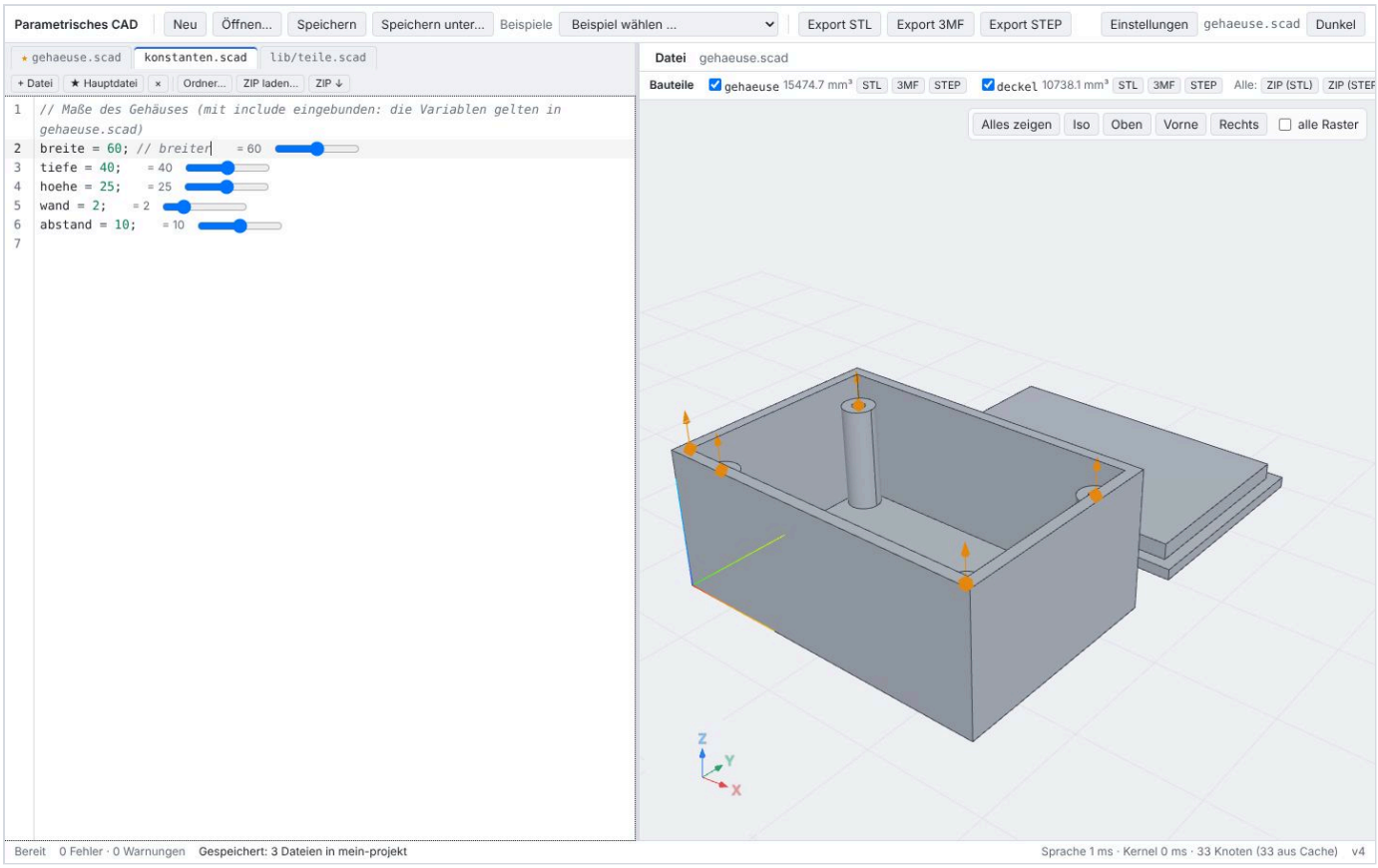
```
// Purpose: project made of several files – dimensions via include (konstanten.scad), parts from
// a library via use (lib/teile.scad), two parts as named instances.
include <konstanten.scad>
use <lib/teile.scad>

gehaeuse: gehaeuse(w = breite, d = tiefe, h = hoehe, t = wand);
translate([0, tiefe + abstand, 0]) deckel: deckel(w = breite, d = tiefe, t = wand);
```

The examples menu opens examples/projekt-gehaeuse/ as a project: gehaeuse.scad (main file, ★), konstanten.scad (via include) and lib/teile.scad (via use). The scene knows the three files with their role; the two named instances at the top level are the parts.

P02 Open folder, save, ZIP

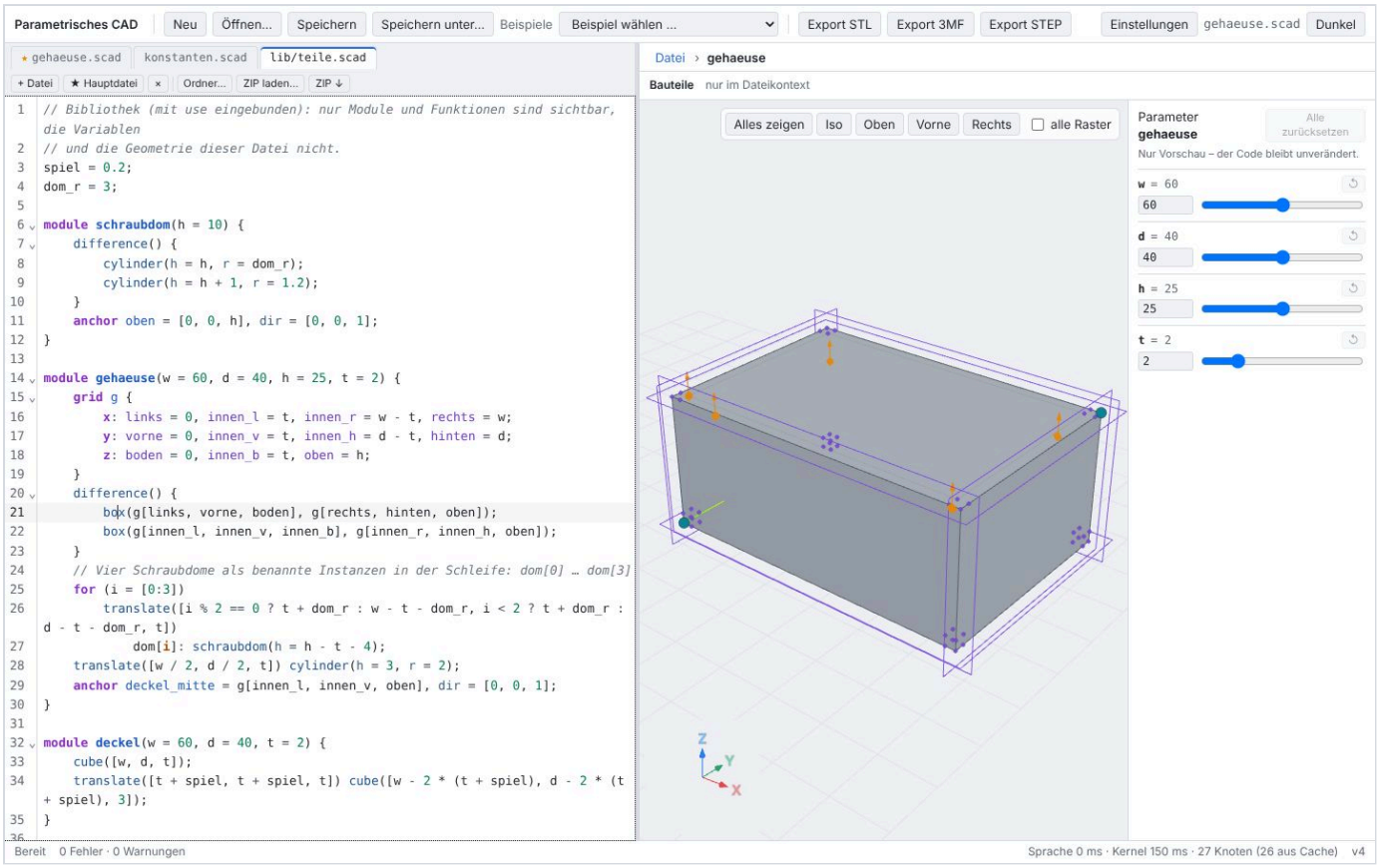
FOLDER "MEIN-PROJEKT" OPENED, KONSTANTEN.SCAD CHANGED AND SAVED



"Folder..." (German UI: „Ordner...“) reads all .scad files of a folder (File System Access API; in the test a simulated folder with a .txt file that is ignored) and selects the main file: the top-level file that no other file includes. Ctrl+S writes all files back to the folder. "ZIP ↓" downloads the project as a ZIP, "Load ZIP..." (German UI: „ZIP laden...“) uploads this ZIP again (the common folder is removed, main file again gehaeuse.scad).

P03 Switch files

LIB/TEILE.SCAD IN THE EDITOR, CURSOR IN MODULE GEHAEUSE (CONTEXT)



CODE

```
module gehaeuse(w = 60, d = 40, h = 25, t = 2) {
    grid g {
        x: links = 0, innen_l = t, innen_r = w - t, rechts = w;
        y: vorne = 0, innen_v = t, innen_h = d - t, hinten = d;
        z: boden = 0, innen_b = t, oben = h;
    }
    difference() {
        box(g[links, vorne, boden], g[rechts, hinten, oben]);
        box(g[innen_l, innen_v, innen_b], g[innen_r, innen_h, oben]);
    }
```

A click on the tab lib/teile.scad shows the library; the cursor in module gehaeuse makes it the context (breadcrumb "File › gehaeuse"), grid and handles belong to the library. Each file keeps its text and its own undo history: Ctrl+Z in the main file does not change the library, Ctrl+Z in the library undoes the change there.

P04 Ctrl+click on include

AFTER CTRL+CLICK ON <KONSTANTEN.SCAD>: THE FILE IS OPEN, VALUES AT THE END OF THE LINE



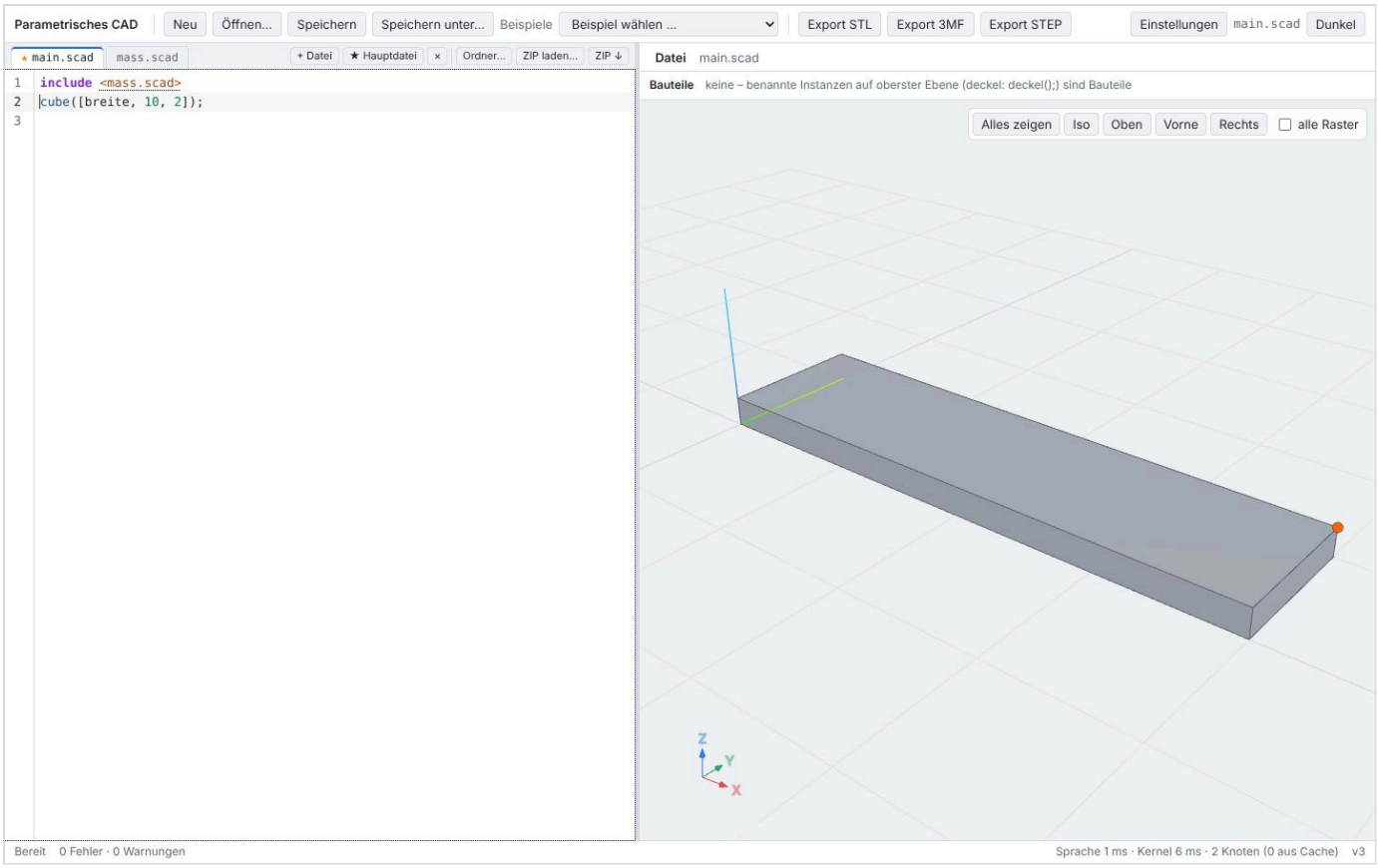
CODE

```
include <konstanten.scad>
use <lib/teile.scad>
```

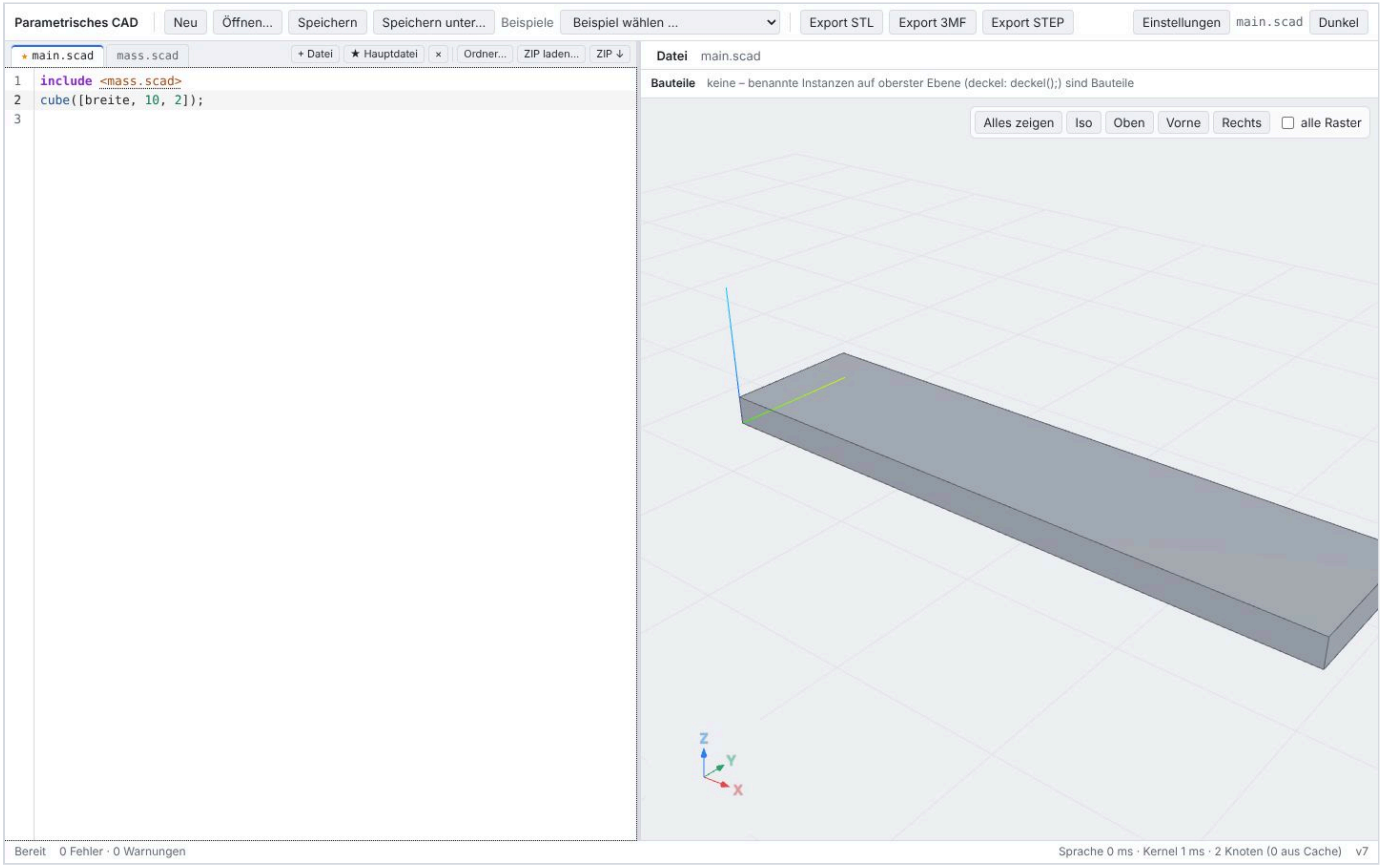
The path in include <…> and use <…> is underlined (tooltip "Ctrl+click: open file", German UI: „Strg+Klick: Datei öffnen“). Ctrl+click opens the file relative to the including file; a plain click only sets the cursor. In konstanten.scad the values of the variables appear at the end of the line, because they apply in the file evaluation via include.

P05 Handle changes the included file

MAIN.SCAD, CURSOR ON CUBE([BREITE, 10, 2]): CORNER HANDLE



AFTER DRAGGING: MASS.SCAD "BREITE = 44;", MAIN.SCAD UNCHANGED



CODE

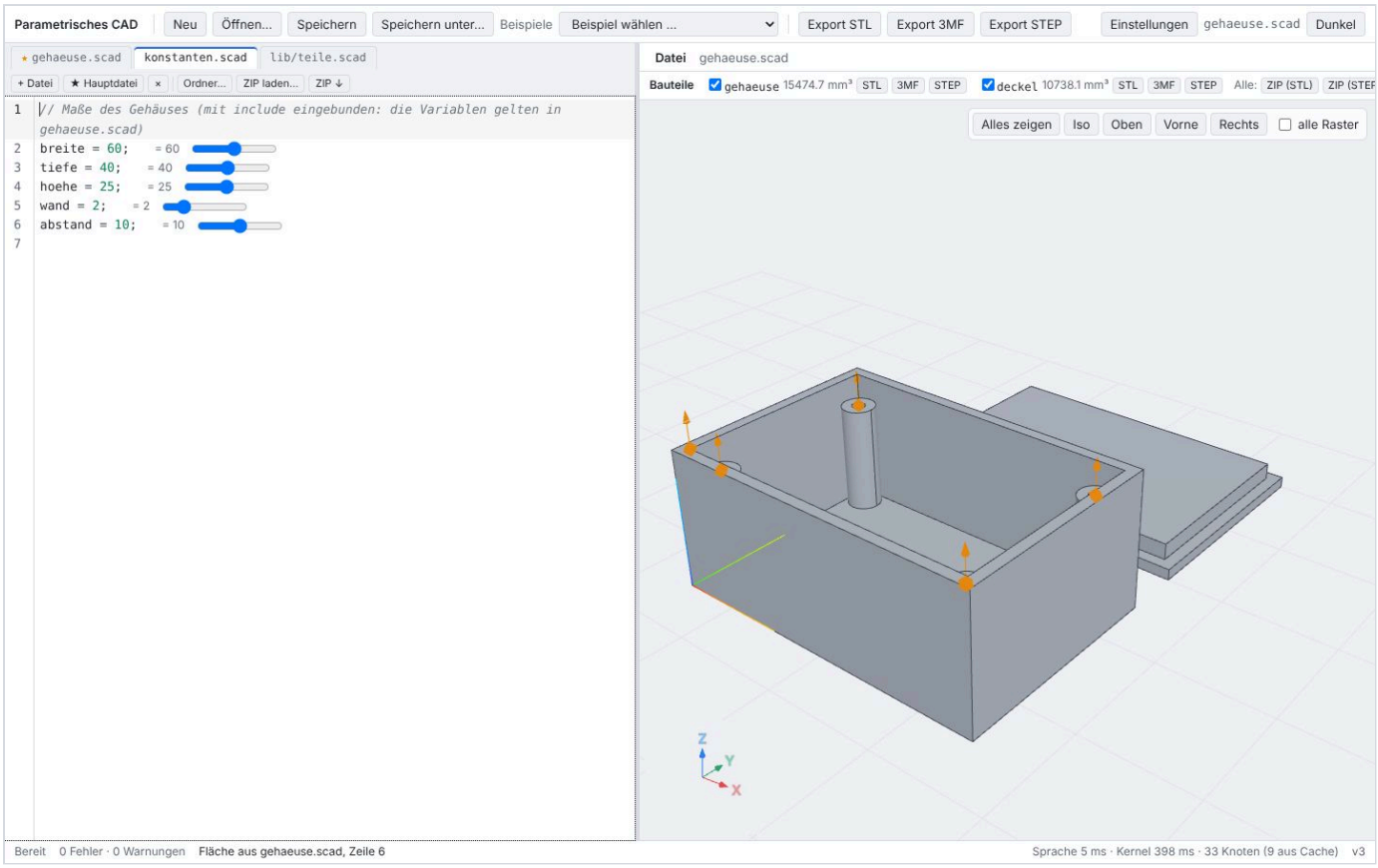
```
// main.scad
include <mass.scad>
```

```
cube([breite, 10, 2]);
// mass.scad
// dimensions
breite = 40;
```

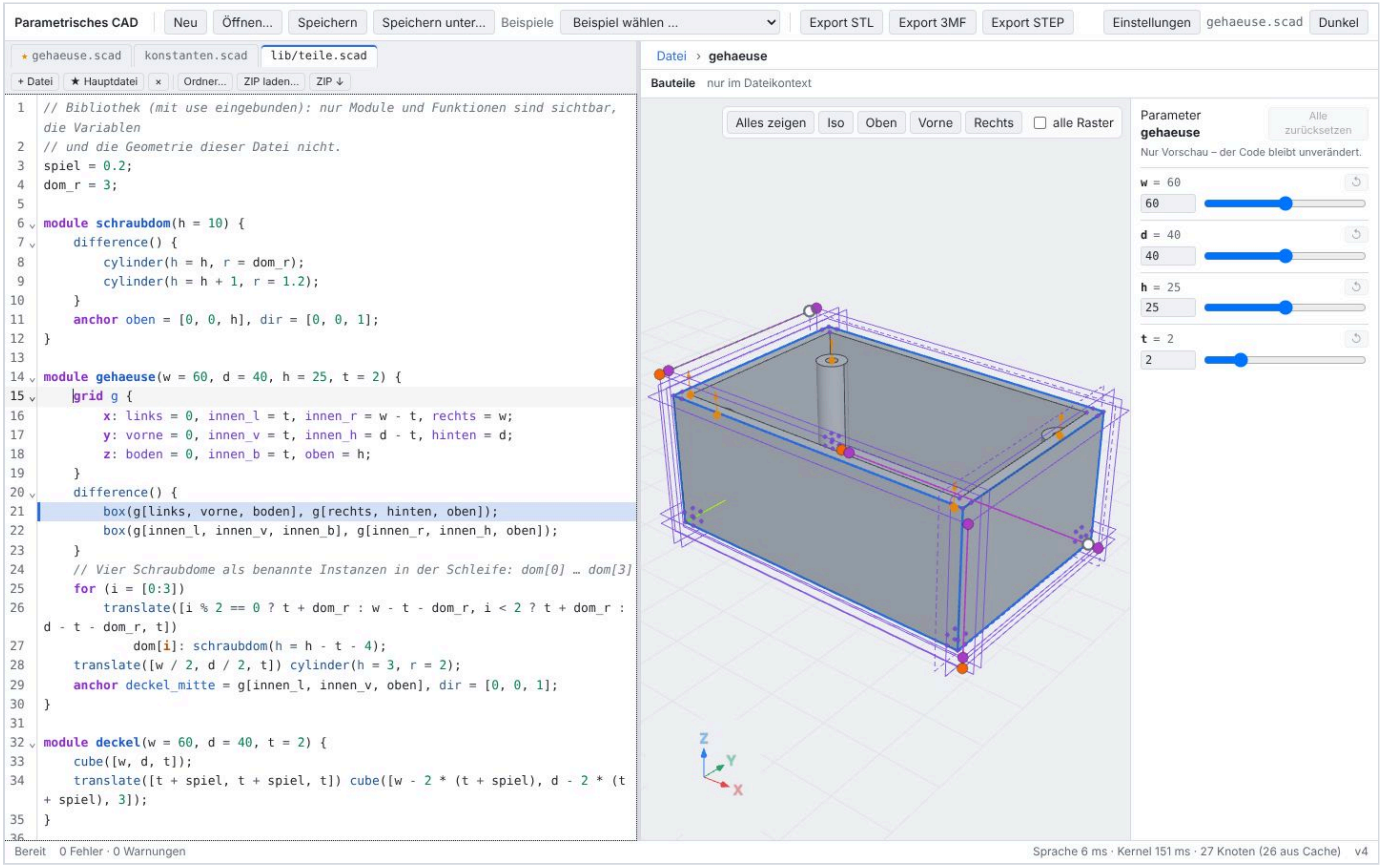
The corner handle of the cuboid goes through the variable breite to the literal in mass.scad. Dragging wrote "breite = 44;" into mass.scad (not into the editor of the main file); the scene followed. The drag is one undo step in mass.scad. Likewise, the radius handle in module gehaeuse of lib/teile.scad changes the literal r = 2 in line 28 of the library, gehaeuse.scad stays the same.

P06 Hover and click across file boundaries

KONSTANTEN.SCAD OPEN, MOUSE OVER THE MODEL: "FACE FROM GEHAEUSE.SCAD, LINE ..."



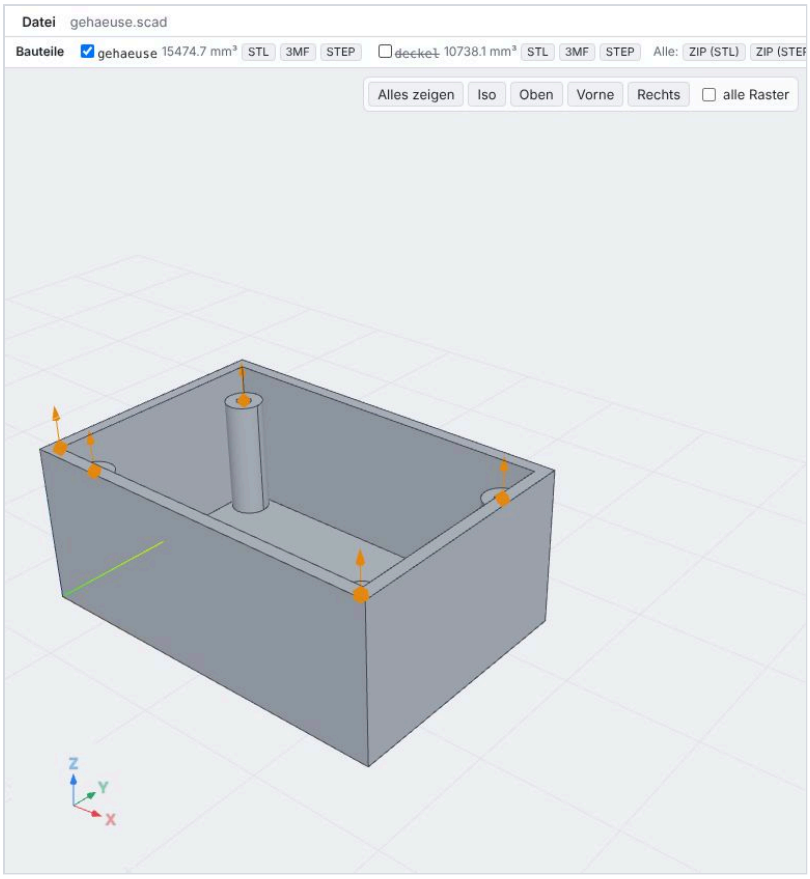
DOUBLE-CLICK ON THE ENCLOSURE: LIB/TEILE.SCAD, MODULE GEHAEUSE



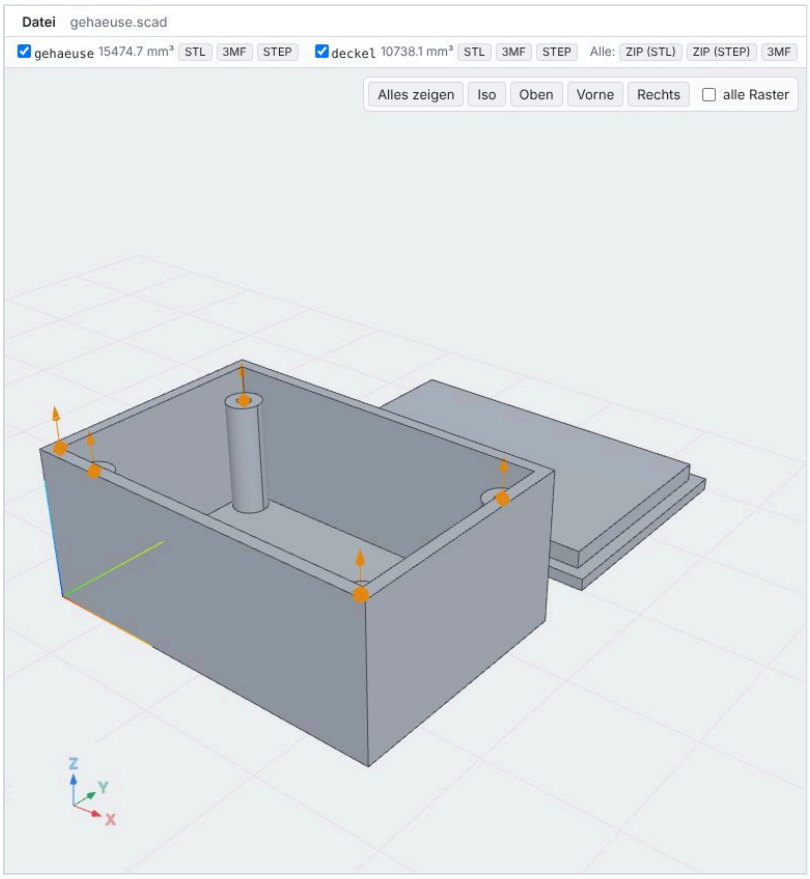
If the line of a face lies in a different file than the one in the editor, the status bar names file and line (German UI: „Fläche aus gehaeuse.scad, Zeile ...“); a click opens the file at that location. A double-click on an instance of a module from another file (gehaeuse from lib/teile.scad) opens that file at the module definition, and the module becomes the context.

P07 Parts list and export per part

LID HIDDEN: ONLY THE ENCLOSURE IS DRAWN



PARTS BAR WITH VOLUME AND EXPORT BUTTONS

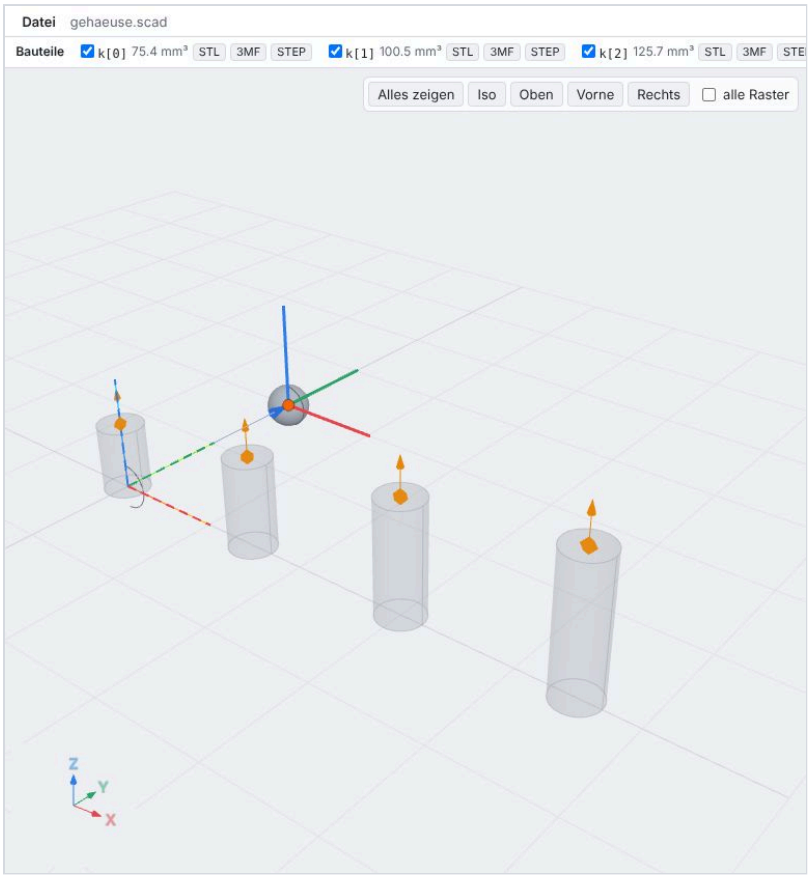


The parts bar lists the parts with their volume (bounding box as tooltip). Without the checkbox, the part is not drawn (the model is assembled from the remaining subtrees). Export: deckel.stl (28 triangles, model coordinates), all parts as a ZIP with one STL per part, one 3MF with one object per part. STEP (exact B-rep, AP242, the same writer as for the whole model):

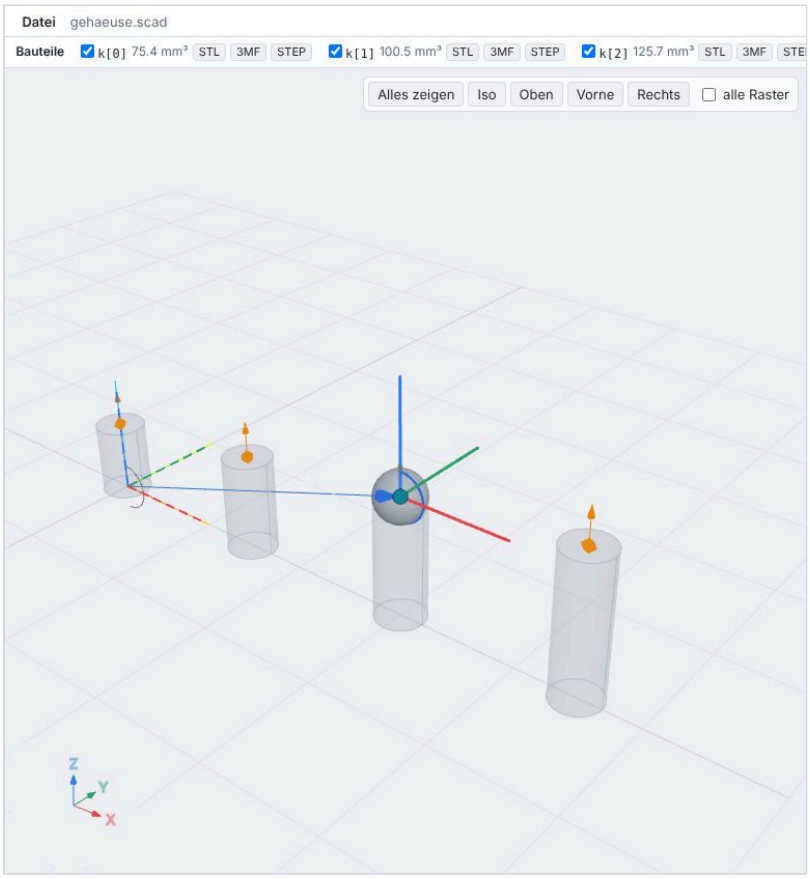
deckel.step 11900 bytes (1 body), model gehaeuse.step 38268 bytes (6 bodies); all parts as a ZIP with one STEP file per part.

P08 Named instances in a loop

CURSOR ON TRANSLATE: FOUR ANCHORS K[0].OBEN … K[3].OBEN AS DIAMONDS



HANDLE RELEASED ON K[2].OBEN



CODE

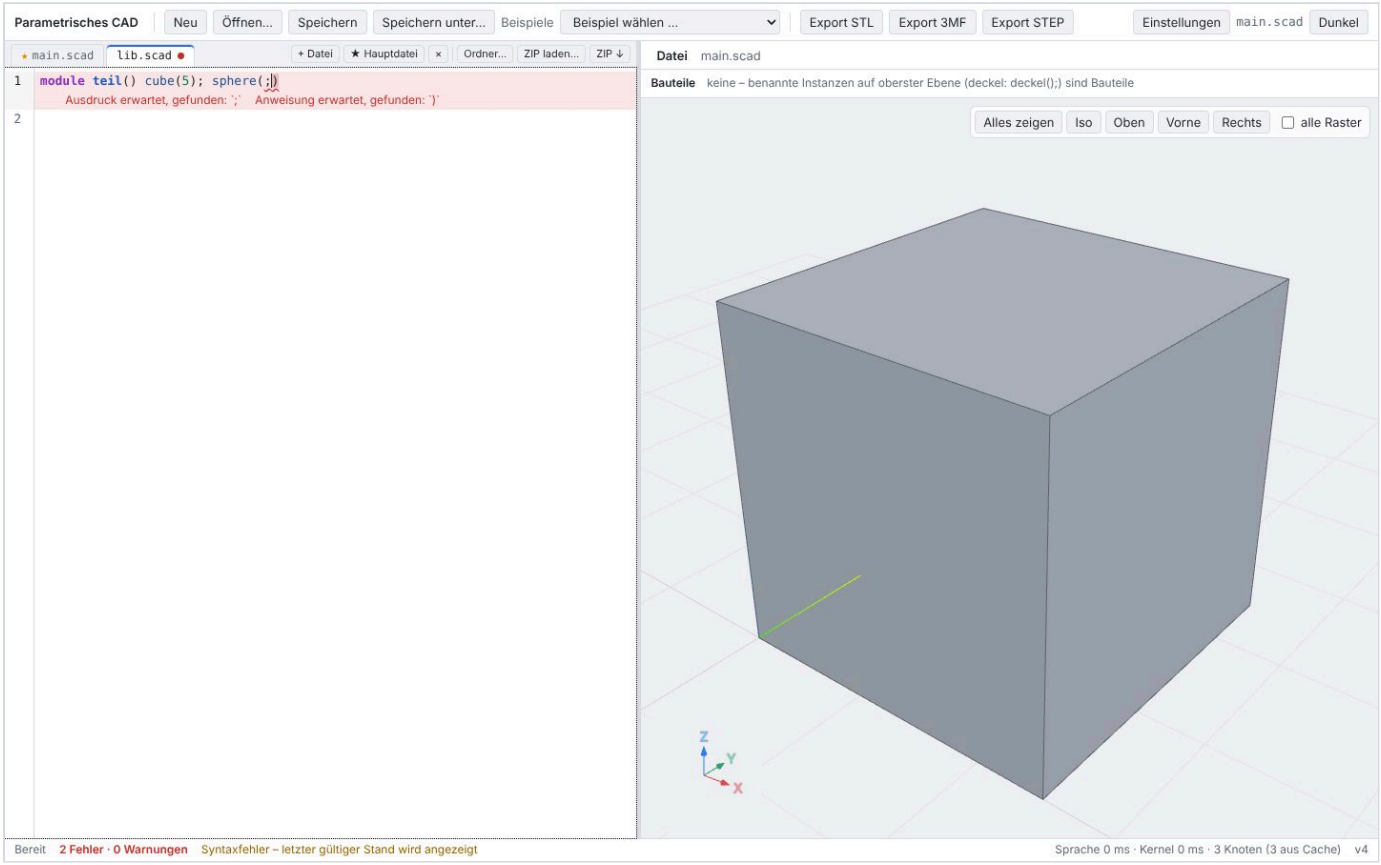
```
module halter(h = 8) {
    cylinder(h = h, r = 2);
```

```
    anchor oben = [0, 0, h], dir = [0, 0, 1];
}
for (i = [0:3]) translate([i * 15, 0, 0]) k[i]: halter(h = 6 + 2 * i);
translate(k[2].oben) sphere(2);
```

for (i = [0:3]) ... k[i]: halter(...) names four instances; their anchors are visible as for named instances (diamonds) and are snap targets, at the top level k[0] ... k[3] are parts. Releasing the translate handle on k[2].oben writes "translate(k[2].oben)"; the sphere sits at [30, 0, 10].

P09 Errors across file boundaries

SYNTAX ERROR IN LIB.SCAD: LINE RED, TAB MARKED; MISSING FILE IN MAIN.SCAD



CODE

```
// main.scad
use <lib.scad>
include <fehlt.scad>
teil();
```

A missing file is an error at the include line of the including file. Diagnostics carry their file: the syntax error in lib.scad appears there (line red) and marks the tab; the view keeps the last valid state.